Document number: P0645R1
Audience: Library Evolution
Author: Victor Zverovich

Reply to:
Victor Zverovich
Lee Howes

2017-02-01

Text Formatting

Revision History

Changes since R0

- Add section Compile-time processing of format strings.
- Separate parsing and formatting in the extension API replacing `format_value` function template with class template `formatter` to allow compile-time processing of format strings.
- Change return type of `format_to` and `vformat_to` to `OutputIterator` in synopsis.
- Remove sections Null-terminated string view and Format string, and replace `basic_cstring_view` with `basic_string_view`.
- Add a link to the implementation in Introduction.
- Add a note regarding time formatting and compatiblity with D0355 "Extending <chrono> to Calendars and Time Zones" [16] to section Extensibility.
- Rename `basic_args` to `basic_format_args`.

- Rename `is_numeric` to `is_arithmetic`.
- Add the `count` function that counts the number of characters and use it to define output ranges.
- Remove `basic_buffer` and section Formatting buffer and replace buffers with output iterators.
- Add Appendix A: Benchmarks.
- Explain the purpose of the type-erased API in more details in the Binary footprint section.
- Add Appendix B: Binary code comparison.
- Add formatting function overloads for `wchar_t` strings.

Introduction

Even with proliferation of graphical and voice user interfaces, text remains one of the main ways for humans to interact with computer programs and programming languages provide a variety of methods to perform text formatting. The first thing we do when learning a new programming language is often writing a "Hello, World!" program that performs simple formatted output.

C++ has not one but two standard APIs for doing formatted output, the `printf` family of functions inherited from C and the I/O streams library (iostreams). While iostreams are usually the recommended way of doing formatted output in C++ for safety and extensibility reasons, `printf` offers some advantages, such as arguably more natural function call API, separation of formatted message and arguments, possibly with argument reordering as a POSIX extension, and often more compact code, both source and binary.

This paper proposes a new text formatting library that can be used as a safe and extensible alternative to the `printf` family of functions. It is intended to complement the existing C++ I/O streams library and reuse some of its infrastructure such as overloaded insertion operators for user-defined types.

Example:

```
string message = fmt::format("The answer is {}.", 42);
```

A full implementation of this proposal is available at https://github.com/fmtlib/fmt/tree/std.

Design

Format string syntax

Variations of the printf format string syntax are arguably the most popular among the programming languages and C++ itself inherits `printf` from C [1]. The advantage of the printf syntax is that many programmers are familiar with it. However, in its current form it has a number of issues:

- Many format specifiers like `hh`, `h`, `l`, `j`, etc. are used only to convey type information. They are redundant in type-safe formatting and would unnecessarily complicate specification and parsing.
- There is no standard way to extend the syntax for user-defined types.
- There are subtle differences between different implementations. For example, POSIX positional arguments [2] are not supported on some systems [6].
- Using `'%'` in a custom format specifier, e.g. for `put_time`-like time formatting, poses difficulties.

Although it is possible to address these issues while maintaining resemblance to the original printf format, this will still break compatibility and can potentially be more confusing to users than introducing a different syntax.

Therefore we propose a new syntax based on the ones used in Python [3], the .NET family of languages [4], and Rust [5]. This syntax employs `'{'` and `'}'` as replacement field delimiters instead of `'%'` and it is described in details in the syntax reference. Here are some of the advantages:

- Consistent and easy to parse mini-language focused on formatting rather than conveying type information
- Extensibility and support for custom format strings for user-defined types
- Positional arguments
- Support for both locale-specific and locale-independent formatting (see Locale support)
- Formatting improvements such as better alignment control, fill character, and binary format

The syntax is expressive enough to enable translation, possibly automated, of most printf format strings. The correspondence between `printf` and the new syntax is given in the following table.

printf	new
-	<
+	+
space	*space*
#	#
0	0
hh	unused
h	unused
l	unused
ll	unused
j	unused
z	unused
t	unused
L	unused
c	c (optional)
s	s (optional)
d	d (optional)
i	d (optional)
o	o
x	x
X	X
u	d (optional)
f	f
F	F
e	e
E	E
a	a
A	A
g	g (optional)
G	G
n	unused
p	p (optional)

Width and precision are represented similarly in `printf` and the proposed syntax with the only difference that runtime value is specified by `*` in the former and `{}` in the latter, possibly with the index of the argument inside the braces.

As can be seen from the table above, most of the specifiers remain the same which simplifies migration from `printf`. Notable difference is in the alignment specification. The proposed syntax allows left, center, and right alignment represented by `'<'`, `'^'`, and `'>'` respectively which is more expressive than the corresponding `printf` syntax. The latter only supports left and right (the default) alignment.

The following example uses center alignment and `'*'` as a fill character:

```
fmt::format("{:*^30}", "centered");
```

resulting in `"**********centered**********"`. The same formatting cannot be easily achieved with `printf`.

In addition to positional arguments, the grammar can be easily extended to support named arguments.

Extensibility

Both the format string syntax and the API are designed with extensibility in mind. The mini-language can be extended for user-defined types and users can provide functions that do parsing, possibly at compile time, and formatting for such types.

The general syntax of a replacement field in a format string is

```
replacement-field ::= '{' [arg-id] [':' format-spec] '}'
```

where `format-spec` is predefined for built-in types, but can be customized for user-defined types. For

example, the syntax can be extended for `put_time`-like date and time formatting

```
time_t t = time(nullptr);
string date = fmt::format("The date is {0:%Y-%m-%d}.", *localtime(&t));
```

by providing a specialization of `formatter` for `tm`:

```
template <>
struct formatter<tm> {
  constexpr fmt::parse_context::iterator parse(fmt::parse_context& ctx);

  template <class FormatContext>
    typename FormatContext::iterator format(const tm& tm,
                                            FormatContext& ctx);
};
```

The `formatter<tm>::parse` function parses the `format-spec` portion of the format string corresponding to the current argument and `formatter<tm>::format` formats the value and writes the output via the iterator `ctx.begin()`.

Note that date and time formatting is not covered by this proposal but formatting facilities provided by D0355 "Extending <chrono> to Calendars and Time Zones" [16] can be easily implemented using this extension API.

The default implementation of `formatter<T>` uses ostream insertion `operator<<` for type `T` if available.

Safety

Formatting functions rely on variadic templates instead of the mechanism provided by `<cstdarg>`. The type information is captured automatically and passed to formatters guaranteeing type safety and making many of the `printf` specifiers redundant (see Format String Syntax). Memory management is automatic to prevent buffer overflow errors common to `printf`.

Locale support

As pointed out in P0067 "Elementary string conversions"[17] there is a number of use cases that do not require internationalization support, but do require high throughput when produced by a server. These include various text-based interchange formats such as JSON or XML. The need for locale-independent functions for conversions between integers and strings and between floating-point numbers and strings has also been highlighted in N4412: Shortcomings of iostreams. Therefore a user should be able to easily control whether to use locales or not during formatting.

We follow Python's approach [3] and designate a separate format specifier `'n'` for locale-aware numeric formatting. It applies to all integral and floating-point types. All other specifiers produce output unaffected by locale settings. This can also have positive effect on performance because locale-independent formatting can be implemented more efficiently.

Positional arguments

An important feature for localization is the ability to rearrange formatting arguments because the word order may vary in different languages [7]. For example:

```
printf("String `%s' has %d characters\n", string, length(string)));
```

A possible German translation of the format string might be:

```
"%2$d Zeichen lang ist die Zeichenkette `%1$s'\n"
```

using POSIX positional arguments [2]. Unfortunately these positional specifiers are not portable [6]. The C++ I/O streams don't support such rearranging of arguments by design because they are interleaved with the portions of the literal string:

```
cout << "String `" << string << "' has " << length(string) << " characters\n";
```

The current proposal allows both positional and automatically numbered arguments, for example:

```
fmt::format("String `{}' has {} characters\n", string, length(string)));
```

with the German translation of the format string:

```
  "{1} Zeichen lang ist die Zeichenkette `{0}'\n"
```

Performance

The formatting library has been designed with performance in mind. It tries to minimize the number of virtual function calls and dynamic memory allocations done per a formatting operation. In particular, if formatting output can fit into a fixed-size array allocated on stack, it should be possible to avoid them altogether by using a suitable API.

The `format_to` function takes an arbitrary output iterator and can be specialized for random access and contiguous iterators for performance reasons as it is done in the reference implementation [14].

The locale-independent formatting can also be implemented more efficiently than the locale-aware one. However, the main goal for the former is to support specific use cases (see Locale support) rather than to improve performance.

See Appendix A: Benchmarks for a small performance comparison of the reference implementation of this proposal to standard formatting facilities.

Binary footprint

In order to minimize binary code size, each formatting function that uses variadic templates can be implemented as a small inline wrapper around its non-variadic counterpart. This wrapper creates a `basic_format_args` object, representing an array of type-erased argument references, with `make_args` and calls the non-variadic function to do the actual work. For example, the `format` variadic function calls `vformat`:

```
string vformat(string_view format_str, format_args args);

template <class... Args>
  inline string format(string_view format_str, const Args&... args) {
    return vformat(format_str, make_args(args...));
  }
```

`basic_format_args` can be implemented as an array of tagged unions. Tags that indicate types of aguments can be combined and passed into a formatting function as a single integer if the number of arguments is small. Since argument types are known at compile time this can be an integer constant and there will be no code generated to compute it at runtime, only to store according to calling conventions.

Given a reasonable optimizing compiler, this will result in a compact per-call binary code, effectively consisting of placing argument pointers (or, possibly, copies for primitive types) and packed tags on stack and calling a formatting function. See Appendix B: Binary code comparison for a specific example.

Exposing the type-erased API rather than making it an implementation detail and only providing variadic functions allows applying the same technique to the user code. For example, consider a simple logging function that writes a formatted error message to `clog`:

```
template <class... Args>
  void log_error(int code, string_view format_str, const Args&... args) {
    clog << "Error " << code << ": " << format(format_str, args...);
  }
```

The code for this function will be generated for every combination of argument types which can be undesirable. However, if we use the type-erased API, there will be only one instance of the logging function while the wrapper function can be trivially inlined:

```
void vlog_error(int code, string_view format_str, format_args args) {
  clog << "Error " << code << ": " << vformat(format_str, args);
}

template <class... Args>
  inline void log_error(int code, string_view format_str, const Args&... args) {
    vlog_error(code, format_str, make_args(args...));
  }
```

The current design allows users to easily switch between the two approaches.

Compile-time processing of format strings

It is possible to parse format strings at compile time with `constexpr` functions which has been

demonstrated in the reference implementation [14] and in [18]. Unfortunately a satisfactory API cannot be provided using existing C++17 features. Ideally we would like to use a function call API similar to the one proposed in this paper:

```
template <class String, class... Args>
  string format(String format_str, const Args&... args);
```

where `String` is some type representing a compile-time format string, for example

```
struct MyString {
  static constexpr string_view value() { return string_view("{}", 2); }
};
```

However, requiring a user to create a new type either manually or via a macro for every format string is not acceptable. P0424R1 "Reconsidering literal operator templates for strings" [15] gives a solution to this problem based on user-defined literal operators. If this or other similar proposal for compile-time strings is accepted into the standard, it should be easy to provide additional formatting APIs that make use of this feature. Obviously runtime checks will still be needed in cases where format string is not known at compile time, but as shown in Appendix A: Benchmarks even with runtime parsing performance can be on par with or better than that of existing methods.

Impact on existing code

The proposed formatting API is defined in the new header `<format>` and should have no impact on existing code.

Proposed wording

Insert a new section in 27.7 [iostream.format].

Header `<format>` synopsis

```
namespace std {
namespace fmt {

template <class Context>
  class basic_arg;

template <class Visitor, class Context>
  see below visit(Visitor&& vis, basic_arg<Context> arg);

template <class Context, class... Args>
  class arg_store;

template <class Context>
  class basic_format_args;

template <class charT>
  class basic_parse_context;

template <class OutputIterator, class charT>
  class basic_context;

using parse_context = basic_parse_context<char>;

using context = basic_context<implementation-defined>;
using wcontext = basic_context<implementation-defined>;

using format_args = basic_format_args<context>;
using wformat_args = basic_format_args<wcontext>;

template <class OutputIterator, class charT>
  using format_args_t = basic_format_args<implementation-defined>

template <class Context, class... Args>
  arg_store<Context, Args...> make_args(const Args&... args);

template <class... Args>
  arg_store<context, Args...> make_args(const Args&... args);

class format_error;

template <class... Args>
```

```
  string format(string_view format_str, const Args&... args);
template <class... Args>
  wstring format(wstring_view format_str, const Args&... args);

string vformat(string_view format_str, format_args args);
wstring vformat(wstring_view format_str, wformat_args args);

template <class OutputIterator, class... Args>
  OutputIterator format_to(OutputIterator out, string_view format_str,
                           const Args&... args);
template <class OutputIterator, class... Args>
  OutputIterator format_to(OutputIterator out, wstring_view format_str,
                           const Args&... args);

template <class OutputIterator>
  OutputIterator vformat_to(OutputIterator out, string_view format_str,
                            format_args_t<OutputIterator, char> args);
template <class OutputIterator>
  OutputIterator vformat_to(OutputIterator out, wstring_view format_str,
                            format_args_t<OutputIterator, wchar_t> args);

template <class... Args>
  size_t count(string_view format_str, const Args&... args);
template <class... Args>
  size_t count(wstring_view format_str, const Args&... args);

template <class T, class charT = char>
struct formatter {
  constexpr typename basic_parse_context<charT>::iterator
    parse(basic_parse_context<charT>& ctx);

  template <class FormatContext>
    typename FormatContext::iterator format(const T& value,
                                            FormatContext& ctx);
};
}  // namespace fmt
}  // namespace std
```

Format string syntax

Format strings contain *replacement fields* surrounded by curly braces `{}`. Anything that is not contained in braces is considered literal text, which is copied unchanged to the output. A brace character can be included in the literal text by doubling: `{{` and `}}`.

The grammar for a replacement field is as follows:

```
replacement-field ::=  '{' [arg-id] [':' format-spec] '}'
arg-id            ::=  integer
integer           ::=  digit+
digit             ::=  '0'...'9'
```

In less formal terms, the replacement field can start with an `arg-id` that specifies the argument whose value is to be formatted and inserted into the output instead of the replacement field. The `arg-id` is optionally followed by a `format-spec`, which is preceded by a colon `':'`. These specify a non-default format for the replacement value.

See also the Format specification mini-language section.

If the numeric `arg-id`s in a format string are 0, 1, 2, ... in sequence, they can all be omitted (not just some) and the numbers 0, 1, 2, ... will be automatically inserted in that order.

Some simple format string examples:

```
"First, thou shalt count to {0}" // References the first argument
"Bring me a {}"                  // Implicitly references the first argument
"From {} to {}"                  // Same as "From {0} to {1}"
```

The `format-spec` field contains a specification of how the value should be presented, including such details as field width, alignment, padding, decimal precision and so on. Each value type can define its own *formatting mini-language* or interpretation of the `format-spec`.

Most built-in types support a common formatting mini-language, which is described in the next section.

A `format-spec` field can also include nested replacement fields in certain position within it. These nested

replacement fields can contain only an argument index; format specifications are not allowed. This allows the formatting of a value to be dynamically specified.

Format specification mini-language

Format specifications are used within replacement fields contained within a format string to define how individual values are presented (see Format string syntax). Each formattable type may define how the format specification is to be interpreted.

Most built-in types implement the following options for format specifications, although some of the formatting options are only supported by arithmetic types.

The general form of a *standard format specifier* is:

```
format-spec ::=  [[fill] align] [sign] ['#'] ['0'] [width] ['.' precision] [type]
fill        ::=  <a character other than '{' or '}'>
align       ::=  '<' | '>' | '=' | '^'
sign        ::=  '+' | '-' | ' '
width       ::=  integer | '{' arg-id '}'
precision   ::=  integer | '{' arg-id '}'
type        ::=  int-type | 'a' | 'A' | 'c' | 'e' | 'E' | 'f' | 'F' | 'g' | 'G' | 'p' | 's'
int-type    ::=  'b' | 'B' | 'd' | 'o' | 'x' | 'X'
```

The `fill` character can be any character other than `'{'` or `'}'`. The presence of a fill character is signaled by the character following it, which must be one of the alignment options. If the second character of `format-spec` is not a valid alignment option, then it is assumed that both the fill character and the alignment option are absent.

The meaning of the various alignment options is as follows:

Option	Meaning
`'<'`	Forces the field to be left-aligned within the available space (this is the default for most objects).
`'>'`	Forces the field to be right-aligned within the available space (this is the default for arithmetic types).
`'='`	Forces the padding to be placed after the sign (if any) but before the digits. This is used for printing fields in the form `+000000120`. This alignment option is only valid for arithmetic types.
`'^'`	Forces the field to be centered within the available space.

Note that unless a minimum field width is defined, the field width will always be the same size as the data to fill it, so that the alignment option has no meaning in this case.

The `sign` option is only valid for arithmetic types, and can be one of the following:

Option	Meaning
`'+'`	Indicates that a sign should be used for both positive as well as negative numbers.
`'-'`	Indicates that a sign should be used only for negative numbers (this is the default behavior).
space	Indicates that a leading space should be used on positive numbers, and a minus sign on negative numbers.

The `'#'` option causes the *alternate form* to be used for the conversion. The alternate form is defined differently for different types. This option is only valid for integer and floating-point types. For integers, when binary, octal, or hexadecimal output is used, this option adds the prefix respective `"0b"` (`"0B"`), `"0"`, or `"0x"` (`"0X"`) to the output value. Whether the prefix is lower-case or upper-case is determined by the case of the type specifier, for example, the prefix `"0x"` is used for the type `'x'` and `"0X"` is used for `'X'`. For floating-point numbers the alternate form causes the result of the conversion to always contain a decimal-point character, even if no digits follow it. Normally, a decimal-point character appears in the result of these conversions only if a digit follows it. In addition, for `'g'` and `'G'` conversions, trailing zeros are not removed from the result.

`width` is a decimal integer defining the minimum field width. If not specified, then the field width will be determined by the content.

Preceding the `width` field by a zero (`'0'`) character enables sign-aware zero-padding for arithmetic types. This is equivalent to a `fill` character of `'0'` with an `alignment` type of `'='`.

The `precision` is a decimal number indicating how many digits should be displayed after the decimal point for a floating-point value formatted with `'f'` and `'F'`, or before and after the decimal point for a floating-point value formatted with `'g'` or `'G'`. For non-arithmetic types the field indicates the maximum field size - in other words, how many characters will be used from the field content. The `precision` is not allowed for integer, character, Boolean, and pointer values.

Finally, the `type` determines how the data should be presented.

The available string presentation types are:

Type	Meaning
`'s'`	String format. This is the default type for strings and may be omitted.
none	The same as `'s'`.

The available character presentation types are:

Type	Meaning
`'c'`	Character format. This is the default type for characters and may be omitted.
none	The same as `'c'`.

The available integer presentation types are:

Type	Meaning
`'b'`	Binary format. Outputs the number in base 2. Using the `'#'` option with this type adds the prefix `"0b"` to the output value.
`'B'`	Binary format. Outputs the number in base 2. Using the `'#'` option with this type adds the prefix `"0B"` to the output value.
`'d'`	Decimal integer. Outputs the number in base 10.
`'o'`	Octal format. Outputs the number in base 8.
`'x'`	Hex format. Outputs the number in base 16, using lower-case letters for the digits above 9. Using the `'#'` option with this type adds the prefix `"0x"` to the output value.
`'X'`	Hex format. Outputs the number in base 16, using upper-case letters for the digits above 9. Using the `'#'` option with this type adds the prefix `"0X"` to the output value.
`'n'`	Number. This is the same as `'d'`, except that it uses the locale to insert the appropriate number separator characters.
none	The same as `'d'`.

Integer presentation types can also be used with character and Boolean values. Boolean values are formatted using textual representation, either `true` or `false`, if the presentation type is not specified.

The available presentation types for floating-point values are:

Type	Meaning
`'a'`	Hexadecimal floating point format. Prints the number in base 16 with prefix `"0x"` and lower-case letters for digits above 9. Uses `'p'` to indicate the exponent.
`'A'`	Same as `'a'` except it uses upper-case letters for the prefix, digits above 9 and to indicate the exponent.
`'e'`	Exponent notation. Prints the number in scientific notation using the letter `'e'` to indicate the exponent.
`'E'`	Exponent notation. Same as `'e'` except it uses an upper-case `'E'` as the separator character.
`'f'`	Fixed point. Displays the number as a fixed-point number.
`'F'`	Fixed point. Same as `'f'`, but converts `nan` to `NAN` and `inf` to `INF`.
`'g'`	General format. For a given precision `p >= 1`, this rounds the number to `p` significant digits and then formats the result in either fixed-point format or in scientific notation, depending on its magnitude. A precision of `0` is treated as equivalent to a precision of `1`.
`'n'`	Number. This is the same as `'g'`, except that it uses the locale to insert the appropriate number separator characters.
none	The same as `'g'`.

The available presentation types for pointers are:

Type	Meaning

`'p'`	Pointer format. This is the default type for pointers and may be omitted.
none	The same as `'p'`.

Formatting functions

```
template <class... Args>
  string format(string_view format_str, const Args&... args);
template <class... Args>
  wstring format(wstring_view format_str, const Args&... args);
```

Effects: The function returns a `string` object constructed from the format string argument `format_str` with each replacement field substituted with the character representation of the argument it refers to, formatted according to the specification given in the field.

Returns: The formatted string.

Throws: `format_error` if `format_str` is not a valid format string.

```
string vformat(string_view format_str, format_args args);
wstring vformat(wstring_view format_str, wformat_args args);
```

Effects: The function returns a `string` object constructed from the format string argument `format_str` with each replacement field substituted with the character representation of the argument it refers to, formatted according to the specification given in the field.

Returns: The formatted string.

Throws: `format_error` if `format_str` is not a valid format string.

```
template <class OutputIterator, class... Args>
  OutputIterator format_to(OutputIterator out, string_view format_str,
                           const Args&... args);
template <class OutputIterator, class... Args>
  OutputIterator format_to(OutputIterator out, wstring_view format_str,
                           const Args&... args);
```

Effects: The function writes to the range `[out, out + fmt::count(format_str, args...))` the format string `format_str` with each replacement field substituted with the character representation of the argument it refers to, formatted according to the specification given in the field.

Returns: The end of the output range.

Throws: `format_error` if `format_str` is not a valid format string.

```
template <class OutputIterator, class... Args>
  OutputIterator vformat_to(OutputIterator out, string_view format_str,
                            format_args_t<OutputIterator, char> args);
template <class OutputIterator, class... Args>
  OutputIterator vformat_to(OutputIterator out, wstring_view format_str,
                            format_args_t<OutputIterator, wchar_t> args);
```

Effects: The function writes to the range `[out, out + size)`, where `size` is the output size, the format string `format_str` with each replacement field substituted with the character representation of the argument it refers to, formatted according to the specification given in the field.

Returns: The end of the output range.

Throws: `format_error` if `format_str` is not a valid format string.

```
template <class... Args>
  size_t count(string_view format_str, const Args&... args);
template <class... Args>
  size_t count(wstring_view format_str, const Args&... args);
```

Effects: The function returns the number of characters in the output of `format(format_str, args...)` without constructing the formatted string.

Returns: The number of characters in the output of `format(format_str, args...)`.

Throws: `format_error` if `format_str` is not a valid format string.

Formatting argument

```
template <class Context>
class basic_arg {
public:
  class handle;

  basic_arg();

  explicit operator bool() const noexcept;

  bool is_arithmetic() const;
  bool is_integral() const;
  bool is_pointer() const;
};
```

The class template `basic_arg` describes objects that store a copy of or a reference to a formatting argument. It is parameterized on a formatting context (see 4.9) and can hold a value of one of the following types:

- `int`
- `unsigned int`
- integral types larger than `int`
- `bool`
- `charT`
- `double`
- floating-point types larger than `double`
- `const charT*`
- `basic_string_view<charT>`
- `const void*`
- `handle` which stores a reference to an object of a user-defined type and a pointer to a formatting function
- `monostate` representing an empty state, i.e. when the object doesn't refer to an argument

where `charT` is `typename Context::char_type`. The value can be accessed via the visitation interface defined in the next section.

```
basic_arg();
```

Effects: Constructs a `basic_arg` object that doesn't refer to an argument.

Postcondition: `!(*this)`.

```
explicit operator bool() const noexcept;
```

Returns: `true` if `*this` refers to an argument, otherwise `false`.

```
bool is_arithmetic() const;
```

Returns: `true` if `*this` represents an argument of an arithmetic type.

```
bool is_integral() const;
```

Returns: `true` if `*this` represents an argument of an integral type ([basic.fundamentall]).

```
bool is_pointer() const;
```

Returns: `true` if `*this` represents an argument of type `const void*`.

Complexity: The invocation of `is_arithmetic`, `is_integral`, and `is_pointer` does not depend on the number of possible value types of a formatting argument.

```
template <class Context>
class basic_arg<Context>::handle {
public:
  void format(Context& ctx);
};

void format(Context& ctx);
```

Effects:

1. Constructs a formatter object `f = typename Context::template formatter_type<T>`, where `T` is the

type of the object referred to by this handle.
2. Parses the format string with `ctx.parse_context().advance_to(f.parse(ctx.parse_context()))`.
3. Formats the object `obj` referred to by this handle with `ctx.advance_to(f.format(obj, ctx))`.

Formatting argument visitation

```
template <class Visitor, class Context>
  see below visit(Visitor&& vis, basic_arg<Context> arg);
```

Requires: The expression in the Effects section shall be a valid expression of the same type, for all alternative value types of a formatting argument. Otherwise, the program is ill-formed.

Effects: Let `value` be the value stored in the formatting argument `arg`. Returns `INVOKE(forward(vis), value);`.

Remarks: The return type is the common type of all possible `INVOKE` expressions in the Effects section. Since exact value types are implementation-defined, visitors should use type traits to handle multiple types.

Complexity: The invocation of the callable object does not depend on the number of possible values types of a formatting argument.

Example:

```
auto uint_value = visit([](auto value) {
  if constexpr (is_unsigned_v<delctype(value)>)
    return value;
  return 0;
}, arg);
```

Class template `arg_store`

```
template <class Context, class... Args>
class arg_store {
public:
  basic_format_args<Context> operator*() const;
};
```

An object of type `arg_store` stores formatting arguments or references to them.

```
basic_format_args<Context> operator*() const;
```

Effects: Constructs a `basic_format_args` object providing access to arguments in `*this`.

Class template `basic_format_args`

```
template <class Context>
class basic_format_args {
public:
  using size_type = implementation-defined;

  basic_format_args() noexcept;

  template <class... Args>
    basic_format_args(const arg_store<Context, Args...>& store);

  basic_arg<Context> operator[](size_type i) const;
};
```

An object of type `basic_format_args` provides access to formatting arguments. Copying a `basic_format_args` object does not copy the arguments.

```
basic_format_args() noexcept;
```

Effects: Constructs an empty `basic_format_args` object.

Postcondition: `!(*this)[0]`.

```
template <class... Args>
  basic_format_args(const arg_store<Context, Args...>& store);
```

Effects: Constructs a `basic_format_args` object that provides access to the agruments in `store`.

```
basic_arg<Context> operator[](size_type i) const;
```

Returns: A `basic_arg` object that represents an argument at index `i` if `i <` the number of arguments. Otherwise, returns an empty `basic_arg` object.

Function template `make_args`

```
template <class Context, class... Args>
  arg_store<Context, Args...> make_args(const Args&... args);
template <class... Args>
  arg_store<context, Args...> make_args(const Args&... args);
```

Effects: The function returns an `arg_store` object that stores references to or copies of formatting arguments `args`.

Returns: The `arg_store` object constructed from the formatting arguments.

Parsing context

```
template <class charT>
class basic_parse_context {
public:
  using char_type = charT;
  using const_iterator = basic_string_view<charT>::const_iterator;
  using iterator = const_iterator;

  explicit constexpr basic_parse_context(basic_string_view<charT> format_str);

  constexpr const_iterator begin() const noexcept;
  constexpr const_iterator end() const noexcept;
  constexpr void advance_to(const_iterator it);

  constexpr unsigned next_arg_id();
  constexpr void check_arg_id(unsigned id);
};
```

The class template `basic_parse_context` is used in the extension API to access the format string range being parsed and the argument counter for automatic indexing.

```
explicit constexpr basic_parse_context(basic_string_view<charT> format_str);
```

Effects: Constructs a parsing context from the format string.

Postconditions: `begin() == format_str.begin()`, `end() == format_str.end()`.

```
constexpr const_iterator begin() const noexcept;
```

Returns: An iterator referring to the first character in the format string range being parsed.

```
constexpr const_iterator end() const noexcept;
```

Returns: An iterator referring to the position one past the last character in the format string range being parsed.

```
void advance_to(iterator it);
```

Effects: Advances the beginning of the parsed format string range to `it`.

Requires: `end()` shall be reachable from `it`.

Postcondition: `begin() == it`.

```
constexpr unsigned next_arg_id();
```

Returns: The next argument ID starting from 0. Each subsequent call to `next_arg_id` gives an ID which is 1 greater than the ID returned from the previous call for the same context object.

Throws: `format_error` if `check_arg_id` has been called earlier which indicates mixing of automatic and manual argument indexing.

```
constexpr void check_arg_id(unsigned id);
```

Throws: `format_error` if `next_arg_id` has been called earlier which indicates mixing of automatic and manual argument indexing.

Formatting context

```
template <class OutputIterator, class charT>
class basic_context {
public:
  using iterator = OutputIterator;
  using char_type = charT;
  using args_type = basic_format_args<basic_context>

  template <class T>
    using formatter_type = formatter<T>;

  basic_parse_context<charT>& parse_context();

  iterator begin();
  void advance_to(iterator it);

  args_type args() const;
};
```

The class `basic_context` represents a formatting context for user-defined types.

```
basic_parse_context<charT>& parse_context();
```

Returns: A reference to the format string parsing context.

```
iterator begin();
```

Returns: An iterator referring to the current position in the output range.

```
void advance_to(iterator it);
```

Effects: Advances the current position in the output range to `it`.

Postcondition: `begin() == it`.

```
args_type args() const;
```

Returns: A copy of the `args_type` object that represents formatting arguments.

User-defined types

If a format string refers to an object of a user-defined type as in

```
   X x;
   string s = format("{}", x);
```

the formatting function first calls `formatter<X>::parse(parse_ctx)` to parse the format specifiers, where `parse_ctx` is a reference to the parsing context. `parse_ctx.begin()` points to the beginning of `format-spec` for the current argument, or to `'}'` if `format-spec` is empty.

The `parse` function should parse `format-spec` and return an iterator past the end of the parsed range.

Then the formatting function calls `formatter<X>::format(x, format_ctx)`, where `x` is a const reference to the argument and `format_ctx` is a reference to the formatting context.

```
template <class T, class charT = char>
struct formatter {
  constexpr typename basic_parse_context<charT>::iterator
    parse(basic_parse_context<charT>& ctx);

  template <typename FormatContext>
    typename FormatContext::iterator format(const T& value,
                                            FormatContext& ctx);
};
```

```
constexpr typename basic_parse_context<charT>::iterator
    parse(basic_parse_context<charT>& ctx);
```

Effects: Returns `ctx.begin()`.

```
template <typename FormatContext>
  typename FormatContext::iterator format(const T& value,
                                          FormatContext& ctx);
```

Effects: The function calls `os << value`, where `os` is an instance of `std::basic_ostream<charT>` with a stream buffer backed by the iterator `ctx.begin()`.

Error reporting

```
class format_error : public runtime_error {
public:
  explicit format_error(const string& what_arg);
  explicit format_error(const char* what_arg);
};
```

The class `format_error` defines the type of objects thrown as exceptions to report errors from the formatting library.

```
format_error(const string& what_arg);
```

Effects: Constructs an object of class `format_error`.

Postcondition: `strcmp(what(), what_arg.c_str()) == 0`.

```
format_error(const char* what_arg);
```

Effects: Constructs an object of class `format_error`.

Postcondition: `strcmp(what(), what_arg) == 0`.

Related work

The Boost Format library [8] is an established formatting library that uses printf-like format string syntax with extensions. The main differences between this library and the current proposal are:

- Syntax: for the reasons descibed in section Format String Syntax this proposal uses a new syntax instead of extending the printf one. This allows much simpler and easier to parse grammar, not burdened by legacy specifiers used to convey type information. For example, Boost Format has two ways to refer to an argument by index and allows but ignores some format specifiers.
- API: Boost Format uses `operator%` to pass formatting arguments while this proposal uses variadic function templates.
- Performance: the implementation of this proposal is several times faster that the implementation of Boost Format on tinyformat benchmarks [9], generates smaller binary code and is faster to compile.

A printf-like Interface for the Streams Library [10] is similar to the Boost Format library but uses variadic templates instead of `operator%`. Unfortunately it hasn't been updated since 2013 and the same arguments about format string syntax apply to it.

The FastFormat library [11] is another well-known formatting library. Similarly to this proposal, FastFormat uses brace-delimited format specifiers, but otherwise the format string syntax is different and the library has significant limitations [12]:

> Three features that have no hope of being accommodated within the current design are:
>
> - Leading zeros (or any other non-space padding)
> - Octal/hexadecimal encoding
> - Runtime width/alignment specification

Formatting facilities of the Folly library [13] are the closest to the current proposal. Folly also uses Python-like format string syntax nearly identical to the one described here. However, the API details are quite different. The current proposal tries to address performance and code bloat issues that are largely ignored by Folly Format. For instance formatting functions in Folly Format are parameterized on all argument types while in this proposal, only the inlined wrapper functions are, which results in much smaller binary code and better compile times.

Implementation

An implementation of this proposal is available in the `std` branch of the open-source fmt library [14].

Acknowledgements

The Format string syntax section is based on the Python documentation [3].

References

[1] *The `fprintf` function. ISO/IEC 9899:2011. 7.21.6.1.*
[2] *fprintf, printf, snprintf, sprintf - print formatted output. The Open Group Base Specifications Issue 6 IEEE Std 1003.1, 2004 Edition.*
[3] *6.1.3. Format String Syntax. Python 3.5.2 documentation.*
[4] *String.Format Method. .NET Framework Class Library.*
[5] *Module `std::fmt`. The Rust Standard Library.*
[6] *Format Specification Syntax: printf and wprintf Functions. C++ Language and Standard Libraries.*
[7] *10.4.2 Rearranging printf Arguments. The GNU Awk User's Guide.*
[8] *Boost Format library. Boost 1.63 documentation.*
[9] *Speed Test. The fmt library repository.*
[10] *A printf-like Interface for the Streams Library (revision 1).*
[11] *The FastFormat library website.*
[12] *An Introduction to Fast Format (Part 1): The State of the Art. Overload Journal #89 - February 2009*
[13] *The folly library repository.*
[14] *The fmt library repository.*
[15] *P0424: Reconsidering literal operator templates for strings.*
[16] *D0355: Extending <chrono> to Calendars and Time Zones.*
[17] *P0067: Elementary string conversions.*
[18] *MPark.Format: Compile-time Checked, Type-Safe Formatting in C++14.*
[19] *Google Benchmark: A microbenchmark support library.*

Appendix A: Benchmarks

To demonstrate that the formatting functions described in this paper can be implemented efficiently, we compare the reference implementation [14] of `format` and `format_to` to `sprintf`, `ostringstream` and `to_string` on the following benchmark. This benchmark generates a set of integers with random numbers of digits, applies each method to convert each integer into a string (either `std::string` or a char buffer depending on the API) and uses the Google Benchmark library [19] to measure timings:

```
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <limits>
#include <sstream>
#include <string>
#include <utility>
#include <vector>

#include <benchmark/benchmark.h>
#include <fmt/format.h>

// Returns a pair with the smallest and the largest value of integral type T
// with the given number of digits.
template <typename T>
std::pair<T, T> range(int num_digits) {
  T first = std::pow(T(10), num_digits - 1);
  int max_digits = std::numeric_limits<T>::digits10 + 1;
  T last = num_digits < max_digits ? first * 10 - 1 :
                                     std::numeric_limits<T>::max();
  return {num_digits > 1 ? first : 0, last};
}

// Generates values of integral type T with random number of digits.
template <typename T>
std::vector<T> generate_random_data(int numbers_per_digit) {
  int max_digits = std::numeric_limits<T>::digits10 + 1;
  std::vector<T> data;
  data.reserve(max_digits * numbers_per_digit);
  for (int i = 1; i <= max_digits; ++i) {
    auto r = range<T>(i);
```

```
    auto value = r.first;
    std::generate_n(std::back_inserter(data), numbers_per_digit, [=]() mutable {
      T result = value;
      value = value < r.second ? value + 1 : r.first;
      return result;
    });
  }
  std::random_shuffle(data.begin(), data.end());
  return data;
}

auto data = generate_random_data<int>(1000);

void sprintf(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data) {
      char buffer[12];
      result += std::sprintf(buffer, "%d", i);
    }
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(sprintf);

void ostringstream(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data) {
      std::ostringstream ss;
      ss << i;
      result += ss.str().size();
    }
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(ostringstream);

void to_string(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data)
      result += std::to_string(i).size();
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(to_string);

void format(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data)
      result += fmt::format("{}", i).size();
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(format);

void format_to(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data) {
      char buffer[12];
      result += fmt::format_to(buffer, "{}", i) - buffer;
    }
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(format_to);

BENCHMARK_MAIN();
```

The benchmark was compiled with clang (Apple LLVM version 9.0.0 clang-900.0.39.2) with `-O3 -DNDEBUG` and run on a macOS system. Below are the results:

```
Run on (4 X 3100 MHz CPU s)
2018-01-27 07:12:00
Benchmark              Time           CPU Iterations
----------------------------------------------------
```

```
sprintf           882311 ns     881076 ns        781
ostringstream    2892035 ns    2888975 ns        242
to_string        1167422 ns    1166831 ns        610
format            675636 ns     674382 ns       1045
format_to         499376 ns     498996 ns       1263
```

The `format` and `format_to` functions show much better performance than the other methods. The `format` function that constructs `std::string` is even 30% faster than the system's version of `sprintf` that uses stack-allocated `char` buffer. `format_to` with a stack-allocated buffer is ~60% faster than `sprintf`.

Appendix B: Binary code comparison

In this section we compare per-call binary code size between the reference implementation that uses techniques described in section Binary footprint and standard formatting facilities. All the code snippets are compiled with clang (Apple LLVM version 9.0.0 clang-900.0.39.2) with `-O3 -DNDEBUG -c -std=c++14` and the resulted binaries are disassembled with `objdump -S`:

```
void consume(const char*);

void sprintf_test() {
  char buffer[100];
  sprintf(buffer, "The answer is %d.", 42);
  consume(buffer);
}

__Z12sprintf_testv:
       0:       55      pushq   %rbp
       1:       48 89 e5        movq    %rsp, %rbp
       4:       53      pushq   %rbx
       5:       48 83 ec 78     subq    $120, %rsp
       9:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      10:       48 8b 00        movq    (%rax), %rax
      13:       48 89 45 f0     movq    %rax, -16(%rbp)
      17:       48 8d 35 37 00 00 00    leaq    55(%rip), %rsi
      1e:       48 8d 5d 80     leaq    -128(%rbp), %rbx
      22:       ba 2a 00 00 00  movl    $42, %edx
      27:       31 c0   xorl    %eax, %eax
      29:       48 89 df        movq    %rbx, %rdi
      2c:       e8 00 00 00 00  callq   0 <__Z12sprintf_testv+0x31>
      31:       48 89 df        movq    %rbx, %rdi
      34:       e8 00 00 00 00  callq   0 <__Z12sprintf_testv+0x39>
      39:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      40:       48 8b 00        movq    (%rax), %rax
      43:       48 3b 45 f0     cmpq    -16(%rbp), %rax
      47:       75 07   jne     7 <__Z12sprintf_testv+0x50>
      49:       48 83 c4 78     addq    $120, %rsp
      4d:       5b      popq    %rbx
      4e:       5d      popq    %rbp
      4f:       c3      retq
      50:       e8 00 00 00 00  callq   0 <__Z12sprintf_testv+0x55>

void format_test() {
  consume(format("The answer is {}.", 42).c_str());
}

__Z11format_testv:
       0:       55      pushq   %rbp
       1:       48 89 e5        movq    %rsp, %rbp
       4:       53      pushq   %rbx
       5:       48 83 ec 28     subq    $40, %rsp
       9:       48 c7 45 d0 2a 00 00 00         movq    $42, -48(%rbp)
      11:       48 8d 35 f4 83 01 00    leaq    99316(%rip), %rsi
      18:       48 8d 7d e0     leaq    -32(%rbp), %rdi
      1c:       4c 8d 45 d0     leaq    -48(%rbp), %r8
      20:       ba 11 00 00 00  movl    $17, %edx
      25:       b9 02 00 00 00  movl    $2, %ecx
      2a:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x2F>
      2f:       f6 45 e0 01     testb   $1, -32(%rbp)
      33:       48 8d 7d e1     leaq    -31(%rbp), %rdi
      37:       48 0f 45 7d f0  cmovneq -16(%rbp), %rdi
      3c:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x41>
      41:       f6 45 e0 01     testb   $1, -32(%rbp)
      45:       74 09   je      9 <__Z11format_testv+0x50>
      47:       48 8b 7d f0     movq    -16(%rbp), %rdi
      4b:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x50>
      50:       48 83 c4 28     addq    $40, %rsp
```

```
      54:       5b      popq    %rbx
      55:       5d      popq    %rbp
      56:       c3      retq
      57:       48 89 c3        movq    %rax, %rbx
      5a:       f6 45 e0 01     testb   $1, -32(%rbp)
      5e:       74 09   je      9 <__Z11format_testv+0x69>
      60:       48 8b 7d f0     movq    -16(%rbp), %rdi
      64:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x69>
      69:       48 89 df        movq    %rbx, %rdi
      6c:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x71>
      71:       66 66 66 66 66 66 2e 0f 1f 84 00 00 00 00 00    nopw    %cs:(%rax,%rax)
```

```
void ostringstream_test() {
  std::ostringstream ss;
  ss << "The answer is " << 42 << ".";
  consume(ss.str().c_str());
}
```

```
__Z18ostringstream_testv:
       0:     55      pushq   %rbp
       1:     48 89 e5        movq    %rsp, %rbp
       4:     41 57   pushq   %r15
       6:     41 56   pushq   %r14
       8:     41 55   pushq   %r13
       a:     41 54   pushq   %r12
       c:     53      pushq   %rbx
       d:     48 81 ec 38 01 00 00    subq    $312, %rsp
      14:     4c 8d b5 18 ff ff ff    leaq    -232(%rbp), %r14
      1b:     4c 8d a5 b0 fe ff ff    leaq    -336(%rbp), %r12
      22:     48 8b 05 00 00 00 00    movq    (%rip), %rax
      29:     48 8d 48 18     leaq    24(%rax), %rcx
      2d:     48 89 8d a8 fe ff ff    movq    %rcx, -344(%rbp)
      34:     48 83 c0 40     addq    $64, %rax
      38:     48 89 85 18 ff ff ff    movq    %rax, -232(%rbp)
      3f:     4c 89 f7        movq    %r14, %rdi
      42:     4c 89 e6        movq    %r12, %rsi
      45:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x4A>
      4a:     48 c7 45 a0 00 00 00 00         movq    $0, -96(%rbp)
      52:     c7 45 a8 ff ff ff ff    movl    $4294967295, -88(%rbp)
      59:     48 8b 1d 00 00 00 00    movq    (%rip), %rbx
      60:     4c 8d 6b 18     leaq    24(%rbx), %r13
      64:     4c 89 ad a8 fe ff ff    movq    %r13, -344(%rbp)
      6b:     48 83 c3 40     addq    $64, %rbx
      6f:     48 89 9d 18 ff ff ff    movq    %rbx, -232(%rbp)
      76:     4c 89 e7        movq    %r12, %rdi
      79:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x7E>
      7e:     4c 8b 3d 00 00 00 00    movq    (%rip), %r15
      85:     49 83 c7 10     addq    $16, %r15
      89:     4c 89 bd b0 fe ff ff    movq    %r15, -336(%rbp)
      90:     48 c7 85 08 ff ff ff 00 00 00 00        movq    $0, -248(%rbp)
      9b:     48 c7 85 00 ff ff ff 00 00 00 00        movq    $0, -256(%rbp)
      a6:     48 c7 85 f8 fe ff ff 00 00 00 00        movq    $0, -264(%rbp)
      b1:     48 c7 85 f0 fe ff ff 00 00 00 00        movq    $0, -272(%rbp)
      bc:     c7 85 10 ff ff ff 10 00 00 00   movl    $16, -240(%rbp)
      c6:     0f 57 c0        xorps   %xmm0, %xmm0
      c9:     0f 29 45 b0     movaps  %xmm0, -80(%rbp)
      cd:     48 c7 45 c0 00 00 00 00         movq    $0, -64(%rbp)
      d5:     48 8d 75 b0     leaq    -80(%rbp), %rsi
      d9:     4c 89 e7        movq    %r12, %rdi
      dc:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0xE1>
      e1:     f6 45 b0 01     testb   $1, -80(%rbp)
      e5:     74 09   je      9 <__Z18ostringstream_testv+0xF0>
      e7:     48 8b 7d c0     movq    -64(%rbp), %rdi
      eb:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0xF0>
      f0:     48 8d 35 dd 10 00 00    leaq    4317(%rip), %rsi
      f7:     48 8d bd a8 fe ff ff    leaq    -344(%rbp), %rdi
      fe:     ba 0e 00 00 00  movl    $14, %edx
     103:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x108>
     108:     be 2a 00 00 00  movl    $42, %esi
     10d:     48 89 c7        movq    %rax, %rdi
     110:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x115>
     115:     48 8d 35 c7 10 00 00    leaq    4295(%rip), %rsi
     11c:     ba 01 00 00 00  movl    $1, %edx
     121:     48 89 c7        movq    %rax, %rdi
     124:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x129>
     129:     48 8d 7d b0     leaq    -80(%rbp), %rdi
     12d:     4c 89 e6        movq    %r12, %rsi
     130:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x135>
     135:     f6 45 b0 01     testb   $1, -80(%rbp)
```

```
139:     48 8d 7d b1     leaq    -79(%rbp), %rdi
13d:     48 0f 45 7d c0  cmovneq -64(%rbp), %rdi
142:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x147>
147:     f6 45 b0 01     testb   $1, -80(%rbp)
14b:     74 09   je      9 <__Z18ostringstream_testv+0x156>
14d:     48 8b 7d c0     movq    -64(%rbp), %rdi
151:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x156>
156:     4c 89 ad a8 fe ff ff    movq    %r13, -344(%rbp)
15d:     48 89 9d 18 ff ff ff    movq    %rbx, -232(%rbp)
164:     4c 89 bd b0 fe ff ff    movq    %r15, -336(%rbp)
16b:     f6 85 f0 fe ff ff 01    testb   $1, -272(%rbp)
172:     74 0c   je      12 <__Z18ostringstream_testv+0x180>
174:     48 8b bd 00 ff ff ff    movq    -256(%rbp), %rdi
17b:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x180>
180:     4c 89 e7        movq    %r12, %rdi
183:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x188>
188:     48 8b 35 00 00 00 00    movq    (%rip), %rsi
18f:     48 83 c6 08     addq    $8, %rsi
193:     48 8d bd a8 fe ff ff    leaq    -344(%rbp), %rdi
19a:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x19F>
19f:     4c 89 f7        movq    %r14, %rdi
1a2:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x1A7>
1a7:     48 81 c4 38 01 00 00    addq    $312, %rsp
1ae:     5b      popq    %rbx
1af:     41 5c   popq    %r12
1b1:     41 5d   popq    %r13
1b3:     41 5e   popq    %r14
1b5:     41 5f   popq    %r15
1b7:     5d      popq    %rbp
1b8:     c3      retq
1b9:     48 89 45 d0     movq    %rax, -48(%rbp)
1bd:     f6 45 b0 01     testb   $1, -80(%rbp)
1c1:     74 3b   je      59 <__Z18ostringstream_testv+0x1FE>
1c3:     48 8b 7d c0     movq    -64(%rbp), %rdi
1c7:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x1CC>
1cc:     eb 30   jmp     48 <__Z18ostringstream_testv+0x1FE>
1ce:     eb 2a   jmp     42 <__Z18ostringstream_testv+0x1FA>
1d0:     48 89 45 d0     movq    %rax, -48(%rbp)
1d4:     f6 45 b0 01     testb   $1, -80(%rbp)
1d8:     74 39   je      57 <__Z18ostringstream_testv+0x213>
1da:     48 8b 7d c0     movq    -64(%rbp), %rdi
1de:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x1E3>
1e3:     f6 85 f0 fe ff ff 01    testb   $1, -272(%rbp)
1ea:     75 30   jne     48 <__Z18ostringstream_testv+0x21C>
1ec:     eb 3a   jmp     58 <__Z18ostringstream_testv+0x228>
1ee:     48 89 45 d0     movq    %rax, -48(%rbp)
1f2:     eb 3c   jmp     60 <__Z18ostringstream_testv+0x230>
1f4:     48 89 45 d0     movq    %rax, -48(%rbp)
1f8:     eb 4d   jmp     77 <__Z18ostringstream_testv+0x247>
1fa:     48 89 45 d0     movq    %rax, -48(%rbp)
1fe:     4c 89 ad a8 fe ff ff    movq    %r13, -344(%rbp)
205:     48 89 9d 18 ff ff ff    movq    %rbx, -232(%rbp)
20c:     4c 89 bd b0 fe ff ff    movq    %r15, -336(%rbp)
213:     f6 85 f0 fe ff ff 01    testb   $1, -272(%rbp)
21a:     74 0c   je      12 <__Z18ostringstream_testv+0x228>
21c:     48 8b bd 00 ff ff ff    movq    -256(%rbp), %rdi
223:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x228>
228:     4c 89 e7        movq    %r12, %rdi
22b:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x230>
230:     48 8b 35 00 00 00 00    movq    (%rip), %rsi
237:     48 83 c6 08     addq    $8, %rsi
23b:     48 8d bd a8 fe ff ff    leaq    -344(%rbp), %rdi
242:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x247>
247:     4c 89 f7        movq    %r14, %rdi
24a:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x24F>
24f:     48 8b 7d d0     movq    -48(%rbp), %rdi
253:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x258>
258:     0f 1f 84 00 00 00 00 00         nopl    (%rax,%rax)
```

The code generated for the `format_test` function that uses the reference implementation of the `format` function described in this proposal is several times smaller than the `ostringstream` code and only 40% larger than the one generated for `sprintf` which is a moderate price to pay for full type and memory safety.

The following factors contribute to the difference in binary code size between `format` and `sprintf`:

- Passing format string as `string_view` instead of `const char*`.

- Using `string` instead of a `char` buffer.
- Preparing the array of formatting arguments.

We can exclude the first two factors from the experiment by mimicking parts of the `sprintf` API:

```
int vraw_format(char* buffer, const char* format, format_args args);

template <typename... Args>
inline int raw_format(char* buffer, const char* format, const Args&... args) {
  return vraw_format(buffer, format, make_args(args...));
}

void raw_format_test() {
  char buffer[100];
  raw_format(buffer, "The answer is {}.", 42);
}

__Z15raw_format_testv:
       0:       55      pushq   %rbp
       1:       48 89 e5        movq    %rsp, %rbp
       4:       48 81 ec 80 00 00 00    subq    $128, %rsp
       b:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      12:       48 8b 00        movq    (%rax), %rax
      15:       48 89 45 f8     movq    %rax, -8(%rbp)
      19:       48 c7 45 80 2a 00 00 00         movq    $42, -128(%rbp)
      21:       48 8d 35 24 12 00 00    leaq    4644(%rip), %rsi
      28:       48 8d 7d 90     leaq    -112(%rbp), %rdi
      2c:       48 8d 4d 80     leaq    -128(%rbp), %rcx
      30:       ba 02 00 00 00  movl    $2, %edx
      35:       e8 00 00 00 00  callq   0 <__Z15raw_format_testv+0x3A>
      3a:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      41:       48 8b 00        movq    (%rax), %rax
      44:       48 3b 45 f8     cmpq    -8(%rbp), %rax
      48:       75 09   jne     9 <__Z15raw_format_testv+0x53>
      4a:       48 81 c4 80 00 00 00    addq    $128, %rsp
      51:       5d      popq    %rbp
      52:       c3      retq
      53:       e8 00 00 00 00  callq   0 <__Z15raw_format_testv+0x58>
      58:       0f 1f 84 00 00 00 00 00         nopl    (%rax,%rax)
```

This shows that passing formatting arguments adds very little overhead and is comparable with `sprintf`.